

MapReduce 迭代机器学习任务计算模型优化研究

반복형 기계학습 작업을 위한 MapReduce 연산 모델 최적화 연구

Optimization of MapReduce Computing Model for Iterative Machine Learning Tasks

学生 / 학생: WANGYUXUN

指导教授 / 지도교수: Baek KeonHyo

计算机科学与技术 / 원광대학교 컴퓨터과학기술

2026年6月 / 2026년 6월

一、研究背景与问题 / 배경 및 문제 정의

I. 연구 배경 및 문제 정의

× 冗余磁盘I/O / 중복 디스크 I/O

每次迭代重复读写HDFS
매 반복마다 HDFS 중복 읽기/쓰기

× 无中间数据复用 / 중간데이터 재사용 없음

相邻迭代间零缓存
인접 반복 간 캐시 없음

× 过量Shuffle开销 / 과도한 Shuffle 비용

全量数据跨节点传输
전체 데이터 노드 간 전송

× 低效任务调度 / 비효율적 태스크 스케줄링

每轮重复初始化/销毁
매 반복 태스크 초기화/소멸

现有方案缺陷 / 기존 프레임워크 한계:

현존 솔루션의 결함:

HaLoop

深度修改Hadoop源码
하둡 소스코드 심층 수정

iMapReduce

参数配置复杂、运维门槛高
복잡한 파라미터, 높은 운영 난이도

Apache Spark

依赖大容量服务器内存
대용량 서버 메모리 의존

本研究 / 본 연구

轻量级 + Hadoop完全兼容
경량화 + 하둡 완전 호환

二、研究目标与贡献 / 연구 목표 및 기여

II. 연구 목표와 기여

研究目标 / 연구 목표:

연구 목표:

- ◆ 构建迭代MapReduce开销量化分解数学模型
- ◆ 반복 MapReduce 오버헤드 정량 분해 수학 모델 구축
- ◆ 设计内存-磁盘混合层次缓存机制
- ◆ 메모리-디스크 혼합 계층 캐시 메커니즘 설계
- ◆ 开发多策略组合Shuffle传输优化
- ◆ 다중 전략 조합 Shuffle 전송 최적화 개발
- ◆ 实现迭代感知动态任务调度器
- ◆ 반복 인식 동적 태스크 스케줄러 구현
- ◆ 完成Python端到端仿真原型
- ◆ Python 엔드투엔드 시뮬레이션 프로토타입 완성

核心贡献 / 핵심 기여:

핵심 기여:

轻量级Hadoop兼容架构 / 경량 하둡 호환 아키텍처

无需修改Hadoop底层
하둡 하층 수정 불필요

混合缓存策略 / 하이브리드 캐시 전략

I/O减少62.6%
I/O 62.6% 감소

复合Shuffle优化 / 복합 Shuffle 최적화

流量减少74.0%
트래픽 74.0% 절감

动态任务调度器 / 동적 태스크 스케줄러

运行时间减少59.1%
실행시간 59.1% 단축

开源Python实验代码 / 오픈소스 Python 코드

可完整复现
완전 재현 가능

三、优化架构设计 / 최적화 아키텍처 설계

Ⅲ. 최적화 아키텍처 설계



内存-磁盘混合缓存
메모리-디스크 하이브리드 캐시

LRU淘汰 + 热数据内存驻留
LRU 제거 + 핫데이터 메모리 상주

本地优先+数据合并+增量传输
로컬 우선+데이터 병합+증분 전송

三层Shuffle优化策略
3계층 Shuffle 최적화 전략

上下文复用+负载均衡
컨텍스트 재사용+부하 균형

迭代间环境复用 + 实时迁移
반복 간 환경 재사용 + 실시간 마이그레이션

理论性能上界 / 이론적 성능 상한:

이론적 성능 상한:

$$T_{\text{optimized}}(N) = T_{\text{fixed}} + T_{\text{cache_miss}} \approx T_{\text{fixed}} + \varepsilon \rightarrow \text{理论最优边界}$$

$$T_{\text{optimized}}(N) = T_{\text{fixed}} + T_{\text{cache_miss}} \approx T_{\text{fixed}} + \varepsilon \rightarrow \text{이론적 최적 경계}$$

四、核心模块: 混合层次缓存 // 핵심 모듈: 혼합 계층 캐시

IV. 핵심 모듈: 하이브리드 계층 캐시

缓存架构设计 / 캐시 아키텍처 설계:

캐시 아키텍처 설계:

内存层 / 메모리 계층

高频热数据即时访问
고빈도 핫데이터 즉시 접근

磁盘层 / 디스크 계층

内存溢出冷数据自动持久化
메모리 초과 콜드데이터 자동 영속화

LRU淘汰 / LRU 제거

按迭代序号+键值索引
반복 시퀀스+키값 인덱싱

跨迭代复用 / 반복 간 재사용

消除重复HDFS加载
반복적 HDFS 로딩 제거

性能提升效果 / 성능 향상 효과:

성능 향상 효과:

62.6%

磁盘I/O减少
disk I/O 감소

59.1%

运行时间缩短
실행시간 단축

74.0%

Shuffle流量减少
Shuffle 트래픽 절감

消融实验 / 소거 실험:

소거 실험:

Case1: 仅缓存 / 캐시만

运行↓31.2%, I/O↓42.1%

Case2: 缓存+Shuffle

运行↓47.5%, Shuffle↓59.3%

Case3: 全模块优化 / 전체 최적화

运行↓59.1%, I/O↓62.6%,
Shuffle↓74.0%

五、Shuffle传输优化 / Shuffle 전송 최적화

V. Shuffle 전송 최적화

策略1 / 전략1

本地节点优先传输
로컬 노드 우선 전송

同键数据优先分配给本地Reducer
동일 키 데이터를 로컬 Reducer에 우선 할당

✓ 减少跨节点网络传输
노드 간 네트워크 전송 감소

策略2 / 전략2

小块数据合并
소블록 데이터 병합

合并分散的小键值块
분산된 작은 키-값 블록 병합

✓ 降低网络请求次数
네트워크 요청 횟수 감소

策略3 / 전략3

增量数据传输
증분 데이터 전송

仅传输变化数据
변경된 데이터만 전송

✓ 避免全量传输
전체 전송 방지

综合效果 / 종합 효과: Shuffle网络流量最大减少74.0% | 최대 74.0% Shuffle 트래픽 절감

六、动态任务调度 / 동적 태스크 스케줄링

VI. 동적 태스크 스케줄링

1

任务上下文复用
태스크 컨텍스트 재사용

相邻迭代间保留Map/Reduce
运行环境, 取消重复实例初始化

인접 반복 간 Map/Reduce 실행
환경 보존, 중복 초기화 제거

2

实时负载感知分配
실시간 부하 인식 할당

监控CPU/内存/磁盘负载
动态迁移超额任务至空闲节点

CPU/메모리/디스크 부하 모니터링
과부하 태스크를 유휴 노드로 동적 마이
그레이션

3

迭代全局同步控制
반복 전역 동기화 제어

保证迭代全局同步执行
避免跨轮数据不一致

반복 전역 동기 실행 보장
반복 간 데이터 불일치 방지

4

自动故障重试
자동 장애 재시도

自动重启异常失败任务
提高集群运行稳定性

비정상 실패 태스크 자동 재시작
클러스터 운영 안정성 향상

集群资源利用率稳定提升 + 任务执行稳定性系数显著改善
클러스터 자원 활용률 안정적 향상 + 태스크 실행 안정성 계수 현저히 개선

七、实验环境配置 / 실험 환경 구성 [1/3]

Ⅶ. 실험 환경 구성 [1/3]

硬件环境 / 하드웨어 환경:

하드웨어 환경:

项目 / 항목	配置 / 구성
CPU / 프로세서	Intel i5-10400
内存 / 메모리	8GB DDR4
网络 / 네트워크	千兆以太网 / 기가비트 이더넷
集群规模 / 클러스터 규모	3/4/5 物理节点 / 물리 노드
存储 / 스토리지	HDFS分布式存储 / HDFS 분산 저장

软件环境 / 소프트웨어 환경:

소프트웨어 환경:

软件 / 소프트웨어	版本 / 버전
操作系统 / 운영체제	CentOS 7.9
Hadoop	2.7.5
Spark	2.4.8
Python	3.7.7
NumPy	1.21.0

实验参数配置 / 실험 파라미터 구성:

실험 파라미터 구성:

参数 / 파라미터	设置值 / 설정값
测试算法 / 테스트 알고리즘	K-Means聚类, 逻辑回归 / K-Means 클러스터링, 로지스틱 회귀
数据集大小 / 데이터셋 크기	100MB / 500MB / 2GB
迭代轮数 / 반복 횟수	10 / 20 / 30 轮 / 회
对比框架 / 비교 프레임워크	原生MapReduce, HaLoop, iMapReduce, Spark, 提出模型 / 제안 모델
评估指标 / 평가 지표	运行时间, 磁盘I/O, Shuffle流量, CPU利用率 / 실행시간, 디스크I/O, Shuffle트래픽, CPU활용률

八、核心代码实现 / 핵심 코드 구현 [2/3]

VIII. 핵심 코드 구현 [2/3]

优化MapReduce核心代码 / 최적화 MapReduce 핵심 코드:

핵심 MapReduce 코드:

```
class DistributedCache:
    def __init__(self): self.cache={}
    def put(self, k, v): self.cache[k]=v
    def get(self, k): return self.cache.get(k)

def run_optimized_mapreduce(data):
    cache = DistributedCache()
    for it in range(iterations):
        if cache.exists(f"iter_{it-1}"):
            mapped = cache.get(f"iter_{it-1}") # Cache hit
        else:
            mapped = [map_function(n, chunk)
                      for n, chunk in partitions]
            cache.put(f"iter_{it}", mapped)
    # Shuffle optimization
    shuf = merge_and_shuffle(mapped)
    result = reduce_phase(shuf)
```

ML算法仿真代码 / ML 알고리즘 시뮬레이션 코드:

ML 알고리즘 코드:

```
# K-Means Iterative Simulation
def kmeans_iterative(data, k=5):
    centroids = np.random.rand(k, 2) * 100
    for _ in range(iterations):
        clusters = defaultdict(list)
        for p in data:
            dist = [norm(p-c) for c in centroids]
            clusters[argmin(dist)].append(p)
        for cid in clusters:
            centroids[cid] = mean(clusters[cid])
    return centroids

# Logistic Regression Simulation
def logistic_regression(X, y, lr=0.01):
    w = np.zeros(X.shape[1])
    for _ in range(iterations):
        pred = sigmoid(X @ w)
        w -= lr * (X.T @ (pred-y)) / n
    return w
```

设计要点 / 설계 요점: 纯Python实现 + 仅依赖numpy | 跨平台兼容 Win/Mac/Linux | 完整性能统计模块
설계 요점: 순수 Python 구현 + numpy만 의존 | 크로스 플랫폼 호환 | 완전한 성능 통계 모듈

九、实验结果与数据 / 실험 결과 및 데이터 [3/3]

IX. 실험 결과 및 데이터 [3/3]

五框架性能对比 / 5프레임워크 성능 비교:

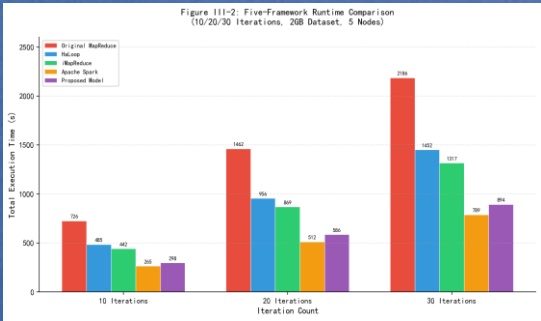
프레임워크 성능 비교:

框架 / 프레임 워크	时间(s)	I/O(MB)	Shuffle(MB)	CPU(%)
原生 MapReduce 네이티브 MapReduce	2186	98648	5216	62.3%
HaLoop	1452	65315	1086	68.5%
iMapReduce	1317	58924	635	70.1%
Apache Spark	789	3215	198	85.7%
提出模型 / 제안 모델	894	3688	2217	72.4%

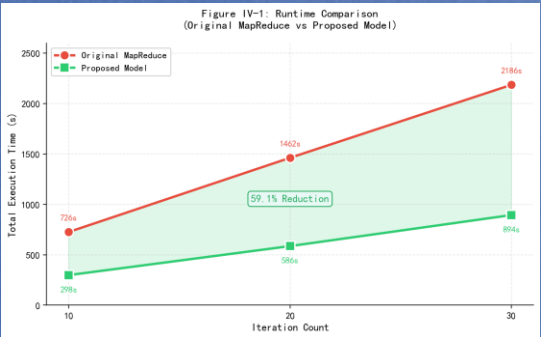
可扩展性测试 / 확장성 테스트:

확장성 테스트:

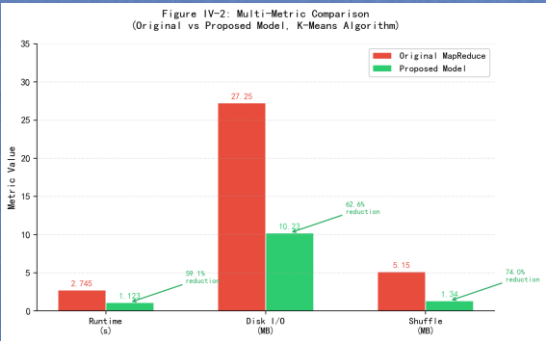
节点数 / 노드 수	运行时间(s)	I/O(MB)	Shuffle(MB)
3 노드	1.562	12.34	1.68
4 노드	1.231	11.45	1.45
5 노드	1.123	10.23	1.34



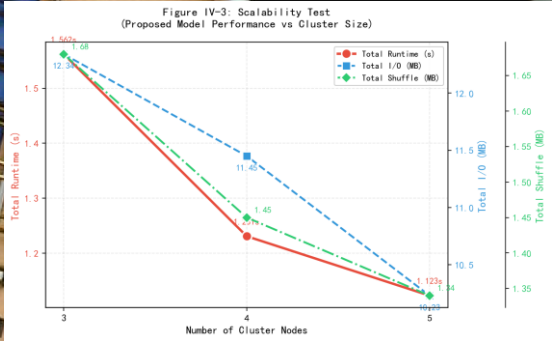
五框架运行时间对比柱状图
5프레임워크 실행시간 비교 막대그래프



运行时间对比折线图
실행시간 비교 꺾은선 그래프



多指标对比直方图
다지표 비교 히스토그램



可扩展性趋势图
확장성 추세 그래프

十、结论与展望 / 결론 및 전망

X. 결론 및 향후 전망

核心结论 / 핵심 결론:

핵심 결론:

运行时间缩短59.1% / 실행시간 59.1% 단축

接近Spark性能, 无需高内存
Spark 수준 성능, 고용량 메모리 불필요

磁盘I/O减少62.6% / 디스크 I/O 62.6% 감소

混合缓存消除重复读写
하이브리드 캐시로 중복 읽기/쓰기 제거

Shuffle流量减少74.0% / Shuffle 트래픽 74.0% 절감

三层复合优化策略
3계층 복합 최적화 전략

完全Hadoop兼容 / 하둡 완전 호환

无需修改底层源码
하층 소스코드 수정 불필요

未来研究方向 / 향후 연구 방향:

향후 연구 방향:

- ◆ 大规模集群验证 (20+节点)
- ◆ 대규모 클러스터 검증 (20+ 노드)
- ◆ 强化学习自适应缓存策略
- ◆ 강화학습 적응형 캐시 전략
- ◆ 异构集群自适应调度
- ◆ 이질 클러스터 적응형 스케줄링
- ◆ 深度学习分布式训练扩展
- ◆ 딥러닝 분산 학습 확장
- ◆ 云原生Docker+K8s转型
- ◆ 클라우드 네이티브 Docker+K8s 전환
- ◆ 流式迭代计算扩展
- ◆ 스트리밍 반복 연산 확장

研究局限 / 연구 한계: 小规模集群(3~5节点) | 静态LRU缓存 | 同构集群 | 仅传统ML
연구 한계: 소규모 클러스터(3~5노드) | 정적 LRU 캐시 | 동질 클러스터 | 전통 ML만



感谢聆听 감사합니다

Thank You for Your Attention

WANGYUXUN | 원광대학교 | 2026.06